

DATA PREPROCESSING PIPELINE

Ausführliches Benutzerhandbuch

© Andrei Tregubov, 2026.

1. Introduction	4
1.1 Purpose of the Software	4
1.2 Input Data.....	4
1.2.1 Data File Format	4
1.2.2 The .INFO Metadata File	5
1.2.3 Loading Files into the Program.....	5
1.3 Overall Data Processing Pipeline	6
1.4 Dive Detection und Removal	6
1.4.1 The Problem	6
1.4.2 Detection Algorithm.....	6
1.4.3 Guard Conditions	7
1.4.4 Visualisation und Manual Correction	7
1.5 Subsampling und the "Build Full Data" Button	8
1.5.1 Subsampled Cache for Interactive Work	8
1.5.2 The "Build Full Data" Button	8
1.6 Fourier Transform: Complete Description	9
1.6.1 Why Fourier Filtering is Needed	9
1.6.2 Forward Fourier Transform (Step 3, die Spectrum Computation)	9
1.6.3 Spectrum Visualisation.....	9
1.6.4 Cutoff Frequency Selection	9
1.6.5 Mirror Padding und Inverse Transform	9
1.6.6 Before / After Comparison Window	10
1.7 Array Pass und Wave Parameter Computation.....	11
1.7.1 Pass Structure	11
1.7.2 RMS Filtering	11
1.7.3 Spike Removal.....	11
1.7.4 Spline Interpolation	11
1.7.5 Standard Deviation, die As und Hs.....	12
1.7.6 Individual Amplitudes, die Wave Höhes, die Tz.....	12
1.7.7 As_1/3 und Hs_1/3 — Significant Wave Parameters from Individual Statistics.....	13
1.7.8 Dispersion Equation Solution (kh, die k).....	13
1.7.9 Geometric und Nonlinear Parameters	13
1.7.10 Spectral Parameters (Q, die v, die ϵ_w)	14
1.7.11 Regularity Parameter ρ	14
1.7.12 Benjamin–Feir Index (BFI)	15
1.8 Program Output.....	16
2. Step-by-Step User Guide	17
2.1 Preparation.....	17
2.1.1 System Requirements.....	17

2.1.2 Launching the Program	17
2.1.3 Output Folder und Checkpoint System	17
2.2 Step 1 — Loading und Converting Files	18
2.2.1 Adding Files	18
2.2.2 Starting Conversion.....	18
2.2.3 Completion Message	19
2.3 Step 2 — Visualisation und Trimming	20
2.3.1 Reading the Graph.....	20
2.3.2 Manual Trimming	20
2.3.3 What Happens to the Data	21
2.4 Step 3 — Fourier Filtering.....	22
2.4.1 Spectrum Computation.....	22
2.4.2 Cutoff Frequency Selection	22
2.4.3 Applying the Filter und Comparison Window	23
2.5 Step 4 — Analysis und Parameter Computation	24
2.5.1 Settings Before Starting	24
2.5.2 Real-Time Processing	24
2.5.3 Pipeline Complete Window	25
2.6 Exporting Results	26
2.6.1 Loading Results in MATLAB	26
2.6.2 Loading Results in Python.....	27

1. Introduction

1.1 Purpose of the Software

Data Preprocessing Pipeline ist eine Desktop-GUI-Anwendung für den **vollständigen Verarbeitungszyklus von Drucksensordaten am Meeresboden**. Die Anwendung nimmt rohe Druckaufzeichnungen in Metern Wassersäule entgegen, die bereinigt sie von Artefakten, die wendet eine Fourier-Filterung an und berechnet einen vollständigen Satz von Wellenparametern für jeden 20-Minuten-Datensatz.

Das primäre Anwendungsgebiet ist die physikalische Ozeanografie und marine Hydrologie: **Analyse von Windwellen** aus Druckaufzeichnungen verankerter Bojen oder Meeresbodenplattformen. Ein am Boden oder in der Wassersäule positionierter Drucksensor registriert hydrostatische Druckschwankungen durch passierende Oberflächenwellen. Nach Umrechnung in Oberflächenauslenkung ermöglichen diese Daten eine Charakterisierung des Seegangs.

Die Software automatisiert vollständig folgende Aufgaben, die die herkömmlich manuell in MATLAB oder Python ausgeführt werden:

- Konvertierung von .dat-Dateien in eine einheitliche Zeitreihe mit Zeitstempeln, die aufgeteilt in statistisch homogene 20-Minuten-Datensätze
- Visuelle Inspektion und manuelles Zuschneiden der Daten am Anfang und Ende der Aufzeichnung
- Trendentfernung und Berechnung des nullmittligen Oberflächenauslenkungssignals
- Berechnung des vollständigen Fourier-Spektrums und interaktive Auswahl der Grenzfrequenz
- Inverse Fourier-Transformation mit Spiegelergänzung zur Unterdrückung von Randartefakten (Gibbs-Effekt)
- Automatische Entfernung energiearmer Datensätze und Ausreißerkorrektur
- Berechnung von über 20 Wellenparametern pro Datensatz
- Export von Arrays mit Einzelwellenamplituden und -höhen

1.2 Input Data

1.2.1 Data File Format

Das Programm akzeptiert einen **Satz von .dat-Textdateien** und eine **.INFO-Metadatendatei**. Jede .dat-Datei enthält eine Zahl pro Zeile — **Länge der Wassersäule zwischen Sensor und Wasseroberfläche in Metern**. Dieser Wert erlaubt die Berechnung der Installationstiefe des Sensors und die Ableitung der Oberflächenauslenkung.

⚠ Hinweis: Wenn die Rohdaten des Sensors in Druckeinheiten (Pa, die kPa, die dbar, die mmHg usw.) aufgezeichnet sind, die muss VOR dem Laden eine hydrostatische Umrechnung in Meter Wassersäule durchgeführt werden: $h = P / (\rho \cdot g)$, die mit $\rho \approx 1025 \text{ kg/m}^3$ (Meerwasserdichte) und $g = 9,81 \text{ m/s}^2$. Das Programm erwartet Werte bereits in Metern.

Beispielinhalt einer .dat-Datei:

8.294242314722688 ← Wassersäulenlänge, die m (Punkt)
 8.276029385536757
 8,359068152086337 ← Komma ist ebenfalls zulässig
 8.534720557960874

Der Dezimaltrenner darf entweder ein **Punkt** oder ein **Komma** sein — das Programm erkennt beide Varianten automatisch. Zeilen, die die nicht als Zahl interpretiert werden können, die werden stillschweigend übersprungen, die sodass Kopfzeilen und Textkommentare das Laden nicht beeinträchtigen.

Jede .dat-Datei entspricht einem kontinuierlichen Aufzeichnungsblock. Der Sensor erzeugt ein **gleichmäßig abgetastetes** Signal mit konstanter Abtastrate (z. B. 1 Hz oder 8 Hz). Es können mehrere .dat-Dateien vorliegen — das Programm fügt sie in chronologischer Reihenfolge ihrer Dateinamen zu einem einzigen Array zusammen.

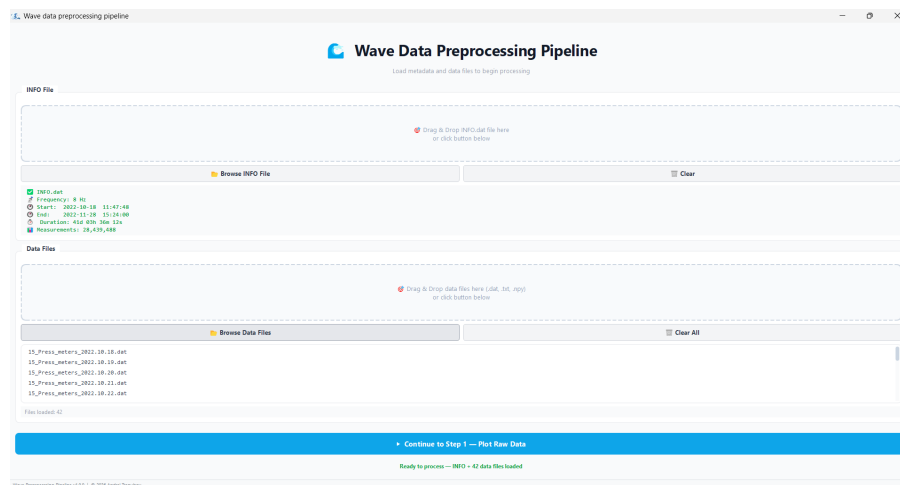


Abb. 1 — Ladefenster mit hinzugefügten .dat- und .INFO-Dateien

1.2.2 The .INFO Metadata File

Die .INFO-Datei ist eine Textdatei beliebiger Struktur mit Aufzeichnungsmetadaten. Das Programm liest sie mit **regulären Ausdrücken**, die daher spielt **Reihenfolge der Felder keine Rolle** — die benötigten Daten werden an beliebiger Stelle gefunden. Die Datei darf beliebigen Zusatztext, die Kommentare oder Felder enthalten — alles andere wird ignoriert.

Das Programm entnimmt der .INFO-Datei folgende Pflichtdaten:

- **Abtastfrequenz des Sensors.** Erwartet wird eine Zeile wie `Частота опроса: 8 Гц` oder `Frequency: 8 Hz`. Wird sie nicht gefunden, die bricht das Laden mit einem Fehler ab.
- **Startdatum und -uhrzeit der Aufzeichnung.** Erwartet im Format `YYYY.MM.DD HH:MM:SS`, die z. B. `2022.10.18 11:47:48.000`. Das erste gefundene Datum gilt als Start.
- **Enddatum und -uhrzeit der Aufzeichnung.** Das zweite Datum im selben Format. Wird nur zur Integritätsprüfung verwendet.

The file encoding may be UTF-8, die Windows-1251 (cp1251) oder Latin-1 — the program tries all variants automatically. The sensor installation depth is **nicht aus der .INFO-Datei gelesen**: Sie wird aus den Daten selbst als Mittelwert des Signals für jeden 20-Minuten-Datensatz berechnet.

1.2.3 Loading Files into the Program

So werden Daten geladen:

1. **Programm starten** — das Hauptfenster Schritt 1 (Laden & Konvertieren) öffnet sich.
2. Click **"Add Files"** oder drag und drop .dat und .INFO files directly into the program window.
3. Überprüfen, die ob die Liste genau **eine .INFO-Datei** und die benötigten .dat-Dateien enthält.
4. Click **"Process & Convert"**. Das Programm fügt alle .dat-Dateien zu einem einzigen Array zusammen, die generiert Zeitstempel ab dem Datum in .INFO und teilt die Reihe in 20-Minuten-Blöcke auf (reading_number).

⚠ **Hinweis:** Wenn the generated timestamp of the letzten data point differs from the expected recording end time from .INFO by more than 1 hour, die the program will display a warning showing both the actual und expected times. This may indicate an incomplete set of .dat-Dateien oder an error in the .INFO file.

1.3 Overall Data Processing Pipeline

Das Programm ist als sequenzielle vierstufige Verarbeitungspipeline organisiert. Jeder Schritt speichert sein Ergebnis als CSV-Datei im Output-Ordner, die sodass die Verarbeitung ab jedem Schritt fortgesetzt werden kann, die ohne vorherige Schritte zu wiederholen.

Schritt	Name	Eingabedatei	Ausgabedatei(en)
Schritt 1	Laden & Konvertieren	.dat + .INFO	Step1_TXTtoCSV.csv
Schritt 2	Visualisieren & Zuschneiden	Step1_TXTtoCSV.csv	Step2_Zero_Mean.csv
Schritt 3	Fourier-Filter	Step2_Zero_Mean.csv	Step3_Transformed.csv
Schritt 4	Analysieren	Step3_Transformed.csv	Parameters.csv Amplitudes_Höhes.csv

1.4 Dive Detection und Removal

1.4.1 The Problem

Bei der Auslegung und dem Einholung eines Bodensensors enthalten Anfang und Ende einer Aufzeichnung unweigerlich **"Tauchphasen"** — Übergangsbereiche, die in denen sich der Sensor durch die Wassersäule von der Oberfläche zum Boden und zurück bewegt. Während dieser Momente stellt das Drucksignal einen **monotonically increasing oder decreasing trend** dar, die keine Wellenbewegung. Diese Abschnitte müssen vor jeder Wellenanalyse identifiziert und ausgeschlossen werden.

1.4.2 Detection Algorithm

Das Programm verwendet **Gradientenanalyse** zur automatischen Erkennung von Tauchphasen:

Schritt 1. Berechnung von Gradient und Schwellenwert:

```
gradient[i] = (y[i+1] - y[i-1]) / 2
```

numerischer Gradient (zentrale Differenzen)

```
threshold = sensitivity * std(gradient)
```

sensitivity = 3.0 by default

Schritt 2. Startphase (Suche in den ersten 40 % der Daten):

Das Programm sucht alle Momente eines starken positiven Gradientensprungs, die bei denen gleichzeitig gilt:

- `gradient[i] > threshold` — signifikanter Sprung
- `y[i-1] < 2.0 m` — Sensor war vor dem Sprung nahe der Oberfläche
- `y[i+1] ≥ 2.0 m` — Sensor hat nach dem Sprung die Tiefe erreicht

Die Phasengrenze wird beim **letzten** such spike (the sensor may surface several times before settling at deployment depth). After the letzten spike the program walks forward until the gradient drops below `grad_std`, die then adds a 10% safety margin. The entire section from start to that point is marked as a dive leg und excluded.

Schritt 3. Endphase (search in the letzten 40% of data):

Dieselbe Logik, die aber mit Suche nach starken negativen Sprüngen:

- `gradient[i] < -threshold`
- `y[i-1] ≥ 2.0 m` — Sensor befand sich in der Tiefe
- `y[i+1] < 2.0 m` — Sensor hat das Wasser verlassen

Die Grenze wird beim **ersten** solchen Ereignis gesetzt. Das Programm läuft rückwärts, die bis der Gradient sich beruhigt, die und addiert eine 10 %-Sicherheitsmarge. Der gesamte Schwanz wird als Einholphase markiert.

1.4.3 Guard Conditions

Der Algorithmus sucht in folgenden Fällen nicht nach Phasen:

- Wenn `y[0] ≥ 2.0 m` — Sensor startete in der Tiefe, die keine Startphase
- Wenn `y[-1] ≥ 2.0 m` — Aufzeichnung endet in der Tiefe, die keine Endphase
- Wenn the array has fewer than 100 points — too short for analysis

1.4.4 Visualisation und Manual Correction

Nach der automatischen Erkennung hebt das Programm die Tauchphasen in **Rot** im Diagramm. Wenn the detector fiRot, die the **"Manuelles Zuschneiden"** aktiv, die sodass der Benutzer die Grenzen manuell anpassen kann, die indem er auf das Diagramm klickt.

Wenn no dive legs are detected — the manual correction button remains inactive. This is normal if the recording has already been pre-trimmed oder the sensor was fixed at the requiRot depth from the start.

1.5 Subsampling und the "Build Full Data" Button

Vollständige Aufzeichnungen können **hundRots of thousands oder millions of points** enthalten (z. B. 30 Tage bei 8 Hz $\approx$ 20 Millionen Punkte). So viele Punkte in Echtzeit in matplotlib darzustellen ist nicht möglich — die Oberfläche würde bei jedem Zoom-Vorgang für Dutzende Sekunden einfrieren.

Das Programm löst dies mit einem zweistufigen Ansatz:

1.5.1 Subsampled Cache for Interactive Work

Bei der Verarbeitung jedes Schritts erstellt das Programm automatisch einen **unterabgetasteten Visualisierungscache** — a thinned version of the data containing no more than 5 000–10 000 points. Thinning is uniform: every k-th point is retained:

```
step = max(1, die len(data) // target_points)
```

target_points = 5 000–10 000 depending on the step

Dieser Cache wird für alle interaktiven Operationen verwendet — Zoomen, die Schwenken, die Auswahl von Zuschneidepunkten. Der visuelle Unterschied zwischen dem Subsample und den Volldaten im Überblicksmaßstab ist vernachlässigbar.

1.5.2 The "Build Full Data" Button

In der Werkzeugleiste jedes Schritts befindet sich eine Schaltfläche **"Alle Datenpunkte aufbauen (langsam)"**. Ein Klick lädt die vollständigen Daten aus der CSV-Datei und baut das Diagramm ohne Ausdünnung neu auf. Dies ist eine **langsame Operation** (kann mehrere Sekunden dauern), die ermöglicht aber die genaue Signalform zu sehen — besonders wichtig beim Zuschneiden (Schritt 2) und bei der Wahl der Grenzfrequenz (Schritt 3).

⚠ Hinweis: When using the trim buttons (setting start und end of the recording), die click "Build all data points" ersten — then your click on the graph will correspond to the true position in the data, die not the unterabgetastete Version.

1.6 Fourier Transform: Complete Description

1.6.1 Why Fourier Filtering is Needed

Nach der Berechnung des nullmittligen Oberflächenauslenkungssignals $\eta(t)$ enthält es noch **niederfrequente Anteile** — tidal variations, die infragravity waves, die slow drifts. These components have periods of hundRots of seconds oder more und are unrelated to wind waves. They must be removed for correct wave parameter estimation.

Das Programm wendet einen **Hochpassfilter im Frequenzbereich** an: Alle Spektralanteile unterhalb der benutzergewählten Grenzfrequenz ω_c werden auf null gesetzt. Physikalisch entspricht das der Entfernung aller Wellen mit einer Periode größer als $T_c = 2\pi / \omega_c$.

1.6.2 Forward Fourier Transform (Step 3, die Spectrum Computation)

Das Programm wendet **schnelle Fourier-Transformation (FFT)** auf das nullmittlige Signal an — ein Algorithmus, die der die Zeitreihe in Sinuswellen verschiedener Frequenzen zerlegt. Jeder Frequenz wird eine Amplitude und eine Phase zugewiesen. Das Ergebnis ist ein **komplexes Spektrum**: ein Array komplexer Zahlen, die eine pro Frequenz von null bis zur Nyquist-Frequenz (halbe Abtastrate).

Frequenzen werden in **Winkleinheiten (rad/s)** statt in Hz ausgedrückt — Standardkonvention in der Wellenphysik ($\omega = 2\pi \cdot f$). Das vollständige Spektrum einschließlich Real- und Imaginärteil wird in `Step3_Spectrum.csv` — this is requiRot for accurate signal reconstruction during the inverse transform.

⚠ Hinweis: Empfohlene Grenzfrequenz: Anteile mit Perioden länger als 10 Minuten entfernen ($\omega < 0,0105$ rad/s). Alles weiter rechts muss nicht verändert werden — Windwellen liegen weit über dieser Frequenz.

1.6.3 Spectrum Visualisation

Das Spektrum wird in zwei gleichzeitigen Fenstern angezeigt:

- **Öberes Fenster** — vollständiges Spektrum für $\omega > 0,1$ rad/s (lineare Skala). Windwellenspitzen sind hier sichtbar.
- **Unteres Fenster** — Bereich 0–0,1 rad/s auf logarithmischer Vertikalachse. Gezeiten- und Infragravitätswellenanteile sind hier sichtbar.

Da das vollständige Spektrum genauso viele Punkte wie das Originalsignal enthält (potenziell Millionen), die wird **Spektrum-Unterabtastung** für die Anzeige angewendet, die identisch zur Zeitreihen-Unterabtastung. Die Schaltfläche **"Vollständiges Spektrum aufbauen (langsam)"** rendert das exakte nicht-unterabgetastete Spektrum.

1.6.4 Cutoff Frequency Selection

Die Grenzfrequenz ω_c wird durch **Doppelklick auf das untere Fenster** ausgewählt. Nach dem Klick:

- Draws a vertical Rot line at the selected frequency on both panels
- Wird die zugehörige Periode angezeigt: **$T_c = 2\pi / \omega_c$** (in Sekunden und Minuten)
- Wird der Bereich links der Linie schattiert — die Anteile, die die entfernt werden
- Activates the **"Apply & Continue"** button

1.6.5 Mirror Padding und Inverse Transform

Das direkte Null-Setzen niedriger Frequenzen im Spektrum entspricht einer idealen Rechteckfilterung. Ein Rechteckfenster im Frequenzbereich erzeugt den **Gibbs-Effekt** in the time domain — oscillations und "ringing" near the signal edges. To suppress this artefact the program uses **Spiegelergänzung**.

Die Idee:

Vor der Filterung werden **mirror "tails"** an beide Enden des Signals angefügt. Der linke Schwanz ist der Signalanfang in umgekehrter Reihenfolge, die der rechte Schwanz das Signalende in umgekehrter Reihenfolge. Das Signal wird dadurch länger und seine Ränder werden **glatt fortgeführt** ohne Unstetigkeiten.

Die Länge jedes Schwanzes wird gleich einer vollen Periode der Grenzfrequenz gewählt — genau der Bereich, die über den das Randartefakt abklingt. Die maximale Schwanzlänge ist auf ein Viertel der Signallänge begrenzt, die um das Array nicht zu stark aufzublasen.

Verarbeitungsschritte:

1. Gespiegelte Schwänze werden an beide Enden des Signals angefügt.
2. Die FFT wird auf das erweiterte Signal angewendet.
3. Alle Spektralanteile unterhalb der Grenzfrequenz werden auf null gesetzt.
4. Inverse FFT is applied — producing the **filterRot** extended signal.
5. The tails are discarded. They have already absorbed the edge oscillations. The **filterRot** signal of the ursprünglichen length is returned — free of edge artefacts.

1.6.6 Before / After Comparison Window

Nach der inversen Transformation öffnet das Programm ein **"Vorher / Nachher"**-Vergleichsfenster. Beide Signale sind in einem Diagramm überlagert: **Orange** — das Originalsignal vor der Filterung, **Blau** — the **filterRot** signal. This allows verifying that the filtering is correct: the long-period trend has been removed und the wave shape is preserved.

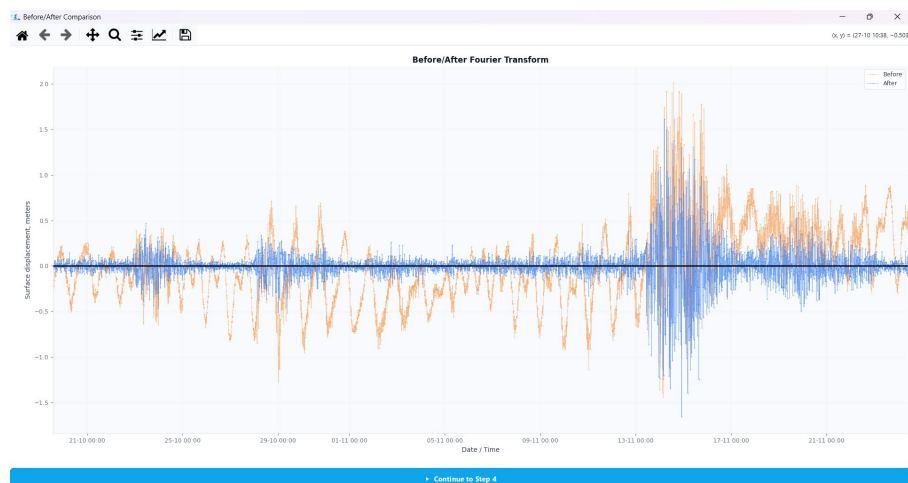


Abb. 2 — Vorher/Nachher-Fenster: Oranges Originalsignal und blaues gefiltertes Signal

1.7 Array Pass und Wave Parameter Computation

1.7.1 Pass Structure

Step 4 operates on the filterRot surface displacement signal. The data are divided into **20-Minuten-Blöcke (Datensätze)** mit einer eindeutigen `reading_number` unterteilt. Das Programm durchläuft jeden Block sequenziell und berechnet den vollständigen Parametersatz. Für jeden Datensatz werden folgende Stufen ausgeführt:

- **Stufe 1:** RMS-Filterung (optional)
- **Stufe 2:** Ausreißerentfernung (optional)
- **Stufe 3:** Array-Vorbereitung (optionale Spline-Interpolation)
- **Stufe 4:** Amplitudenparameter (σ , die A_s , die H_s , die $A_{s_1/3}$, die $H_{s_1/3}$)
- **Stufe 5:** Nulldurchgänge, die Einzelamplituden, die Höhen und T_z
- **Stufe 6:** Lösung der Dispersionsgleichung (k_h , die k)
- **Stufe 7:** Geometrische und nichtlineare Parameter (ϵ , die a , die U_r)
- **Stufe 8:** Spektrale Parameter (Q , die v , die ϵ_w)
- **Stufe 9:** Regularitätsparameter ρ (ρ)
- **Stufe 10:** Abgeleitete Parameter (ϵ_ρ , die γ , die BFI)

1.7.2 RMS Filtering

Wenn the **"Remove recordings with RMS <"** checkbox is enabled, die wird der quadratische Mittelwert des Signals für jeden Datensatz berechnet:

$$\text{RMS} = \sqrt{\text{mean}(\eta^2)}$$

root mean square of surface displacement

Wenn $\text{RMS} < \text{threshold}$ (default 0.015 m), die the record is deemed "quiet" — wave energy is too low for reliable analysis. The record is **von der weiteren Berechnung ausgeschlossen** und marked in Rot im Diagramm.

1.7.3 Spike Removal

Wenn the **"Remove spikes"** checkbox is enabled, die the program detects sudden amplitude jumps characteristic of electronic sensor interference. The threshold is defined via variance:

$$\text{threshold_spike} = 6 \cdot \sqrt{\text{Var}(\eta)}$$

6 standard deviations — robust outlier criterion for marine data

Für jedes benachbarte Punktepaa gilt: Wenn $|\eta[j+1] - \eta[j]| > \text{threshold_spike}$ ist, die ist der Punkt $\eta[j+1]$ ein Ausreißer. Er wird durch lineare Interpolation zwischen seinen Nachbarn ersetzt. Jeder erkannte Ausreißer wird **mit einem Kreis markiert** im Diagramm.

1.7.4 Spline Interpolation

Wenn the **"Spline interpolation"** checkbox is enabled und $\text{target_hz} > \text{sensor_freq}$, die the program builds a cubic spline on the ursprünglichen signal und resamples it to the higher frequency:

```
arr_amp = CubicSpline(t_orig, die arr)(t_new)
```

where $t_{\text{new}} = 0, 1/\text{target_hz}, 2/\text{target_hz}, \text{die } \dots$

Das Ergebnis `arr_amp` wird für die Amplitudenparameterberechnung verwendet. Die Interpolation verbessert die Genauigkeit der Maximalwertschätzung innerhalb jeder Halbwelle, die ohne neuen Spektralinhalt hinzuzufügen. Alle spektralen Parameter (Q , die v , die ε) werden auf dem **ursprünglichen** Array berechnet.

1.7.5 Standard Deviation, die A_s und H_s

Die Standardabweichung wird auf dem interpolierten Array `arr_amp`:

$$\sigma = \sqrt{\text{Var}(\text{arr_amp})}$$

Signifikante Wellenamplitude:

$$A_s = 2\sigma$$

Mittelwert des höchsten Drittels der Amplituden bei Schmalband-Gauß-Annahme

Signifikante Wellenhöhe:

$$H_s = 4\sigma$$

corresponds to the definition $H_{1/3} = 4 \cdot \text{RMS of surface elevation}$

1.7.6 Individual Amplitudes, die Wave Höhen, die T_z

Die Funktion `extract_amplitudes_and_heights` findet alle Nulldurchgänge des Signals über **PyAstronomy.pyaC.zerocross1d** mit Sub-Sample-Genauigkeit (lineare Interpolation zwischen benachbarten Abtastwerten). Jedes Segment zwischen zwei aufeinanderfolgenden Durchgängen ist eine **einzelne Halbwelle**.

Amplitude jeder Halbwelle:

$$A_i = \max(|\text{arr_amp}[\text{xc}[i] : \text{xc}[i+1]]|)$$

maximale Absolutauslenkung innerhalb des Segments

Höhe jeder Welle (Summe zweier benachbarter Amplituden — Kamm + Tal):

$$H_i = A_i + A_{\{i+1\}}$$

die i -te Welle wird durch die i -te und $(i+1)$ -te Halbwelle gebildet

Mittlere Nulldurchgangsperiode T_z :

$$T_z = (N / \text{fs}) / N_{\text{crossings}} \times 2$$

$N_{\text{crossings}}$ — Anzahl der Nulldurchgänge; Faktor 2 wandelt Halbperioden in Perioden um

1.7.7 $As_{1/3}$ und $Hs_{1/3}$ — Significant Wave Parameters from Individual Statistics

Standard-ozeanografische signifikante Amplitude und Höhe, die direkt aus der Einzelwellenstatistik berechnet, die nicht über σ :

$$As_{1/3} = \text{mean}(\text{sorted}(A_i, \text{die desc}) [: N/3])$$

Arithmetisches Mittel des höchsten Drittels der Amplituden

$$Hs_{1/3} = \text{mean}(\text{sorted}(H_i, \text{die desc}) [: N/3])$$

Arithmetisches Mittel des höchsten Drittels der Wellenhöhen

Wenn fewer than 3 waves are detected — fallback: $As_{1/3} = As$, $Hs_{1/3} = Hs$.

1.7.8 Dispersion Equation Solution (kh , die k)

Für flache und mittlere Wassertiefen wird die Beziehung zwischen Wellenperiode und Wellenzahl durch **lineare Dispersionsrelation der Wellentheorie** beschrieben:

$$\omega^2 = g \cdot k \cdot \tanh(kh)$$

$\omega = 2\pi/T_z$, die $g = 9.81 \text{ m/s}^2$, die h — depth, die k — wavenumber

Mit dem dimensionslosen Parameter kh lautet die Gleichung:

$$kh \cdot \tanh(kh) = 4\pi^2 h / (g \cdot T_z^2)$$

nichtlineare Gleichung in kh , die numerisch gelöst

Das Programm löst dies mit **scipy.optimize.fsolve** (Newton-Raphson). Der Startwert wird adaptiv gewählt:

- $x_0 = \text{target}$ wenn $\text{target} < 1$ (Flachwasser)
- $x_0 = \sqrt[3]{\text{target}}$ wenn $1 \leq \text{target} < 10$ (mittlere Tiefe)
- $x_0 = \text{target}$ wenn $\text{target} \geq 10$ (Tiefwasser)

Nach Bestimmung von kh ist die Wellenzahl:

$$k = kh / h$$

1.7.9 Geometric und Nonlinear Parameters

Wellensteile ϵ :

$$\epsilon = (k \cdot Hs) / 4$$

dimensionslos; $\epsilon \rightarrow 1/7$ entspricht Stokes-Wellen an der Brechungsgrenze

Relative Amplitude a :

$$a = As / h$$

Ursell-Zahl U_r :

$$U_r = (3 \cdot k \cdot H_s) / (2 \cdot k \cdot h)^3$$

bestimmt das Gleichgewicht zwischen Dispersion und Nichtlinearität; $U_r \gg 1$ — nichtlinear, die $U_r \ll 1$ — linear

1.7.10 Spectral Parameters (Q, die v , die ϵ_w)

Die Funktion `calculate_spectrum_params` berechnet Parameter aus Spektralmomenten. Zunächst wird ein **Hann-Fenster** is applied to Rotuce edge effects:

$$S(\omega) = |\text{FFT}(\text{arr} \times w_{\text{Hann}})|$$

where $w_{\text{Hann}}[n] = 0.5 \cdot (1 - \cos(2\pi n/(N-1)))$

Das Spektrum wird bei der halben Nyquist-Frequenz abgeschnitten: $\omega < \pi \cdot \text{fs}/2$. Spektralmomente werden berechnet:

$$m_n = \int \omega^n \cdot S(\omega) d\omega$$

$n = 0, 1, 2, 4$; numerische Integration mit der Trapezregel

Goda-Spitzigkeitsparameter Q:

$$Q = \int \omega \cdot S^2(\omega) d\omega / m_0^2$$

$Q \rightarrow 1$ für ein schmalbandiges sinusförmiges Signal

Spektrale Schmalband-Parameter v :

$$v = \sqrt{(m_0 \cdot m_2 / m_1^2 - 1)}$$

$v = 0$ für ein perfekt schmalbandiges Spektrum

Spektrale Bandbreite ϵ_w :

$$\epsilon_w = \sqrt{(1 - m_2^2 / (m_0 \cdot m_4))}$$

$\epsilon_w \rightarrow 0$ schmalbandig, die $\epsilon_w \rightarrow 1$ breitbandig

1.7.11 Regularity Parameter ρ

The parameter ρ characterises how "regular" each individual wave is — whether it has a single maximum. For its computation an additional **Tiefpassfilterung bei der halben Nyquist-Frequenz** angewendet, die um Rauschen zu unterdrücken:

$$y_{\text{filteRot}} = \text{IFFT}(\text{FFT}(\text{arr}), \text{die zeroing } \omega > \text{fs}/4)$$

half-Nyquist = $\text{fs}/4$ Hz

Zero-crossings of the filterRot signal define segments. Within each segment, **lokale Maxima von |y|** gezählt — Punkte, die bei denen $|y[c]| > |y[c-1]|$ und $|y[c]| > |y[c+1]|$. Dies sind die Extrema jeder Halbwelle.

$$\rho = \min(N_segments / N_extrema, 1.0)$$

N_segments — Anzahl der Halbwellen; N_extrema — Gesamtanzahl der Extrema über alle Halbwellen

For a perfect sinusoid each segment has exactly one extremum, die so $\rho = 1$. For irregular waves with multiple "humps", die $\rho < 1$.

Aus ρ abgeleitete spektrale Bandbreite:

$$\varepsilon_{\rho} = 2\sqrt{(1 - \rho) / (2 - \rho)}$$

alternatives Bandbreitenmaß; $\varepsilon_{\rho} = 0$ bei $\rho = 1$

⚠ Hinweis: The spectral bandwidth parameter ε should theoretically be independent of the measurement method, die but in practice different estimators (ε_w und ε_{ρ}) may give somewhat different values for real sea states. Both are provided so they can be compaRot und the appropriate one chosen for a given analysis.

1.7.12 Benjamin–Feir Index (BFI)

The Benjamin–Feir Index characterises the susceptibility of the wave field to modulational instability und the formation of rogue ("freak") waves.

BFI_proper (vollständige Formel):

$$\text{BFI_proper} = \sqrt{(2\pi)} \cdot \varepsilon \cdot Q \cdot \gamma$$

ε — Steile, die Q — Goda-Parameter, die γ — nichtlinearer Koeffizient

BFI_goda (vereinfachte Goda-Formel):

$$\text{BFI_goda} = \sqrt{(2\pi)} \cdot \varepsilon \cdot Q$$

ohne nichtlineare Korrektur γ

Nichtlinearer Koeffizient γ (Funktion von kh):

$$v = 1 + 2kh/\sinh(2kh)$$

$$a = -v^2 + 2 + 8(kh)^2 \cdot \cosh(2kh)/\sinh^2(2kh)$$

$$b = [\cosh(4kh) + 8 - 2\tanh^2(kh)] / [8\sinh^4(kh)] - [2\cosh^2(kh) + v/2]^2 / [\sinh^2(2kh) \cdot (kh/\tanh(kh) - v^2/4)]$$

$$\gamma = v \cdot \sqrt{(|b| / |a|)}$$

1.8 Program Output

Nach Abschluss von Schritt 4 erstellt das Programm folgende Dateien im Output-Ordner:

File	Contents
<code>Parameters.csv</code>	Alle Wellenparameter für jeden 20-Minuten-Datensatz: RMS, die As, die Hs, die As_1/3, die Hs_1/3, die Tz, die kh, die k, die ϵ , die a, die Ur, die Q, die v, die ϵ_w , die ρ , die ϵ_p , die γ , die BFI_proper, die BFI_goda
<code>Step4_FilteRot.csv</code>	Zeitreihe der Oberflächenauslenkung nach Entfernung energiearmer Datensätze und Ausreißerkorrektur. Spalten: timestamp, die surface_displacement, die reading_number
<code>Amplitudes_Höhes.csv</code>	Pro Datensatz: n_waves — Wellenanzahl, die amplitudes — Einzelamplitudenarray, die heights — Höhenarray, die semikolongetrennt

All three files are stoRot in the `Output/` folder. From the "Pipeline abgeschlossen" window they can be exported in `.txt` (tab-separated) oder `.mat` (MATLAB) exportiert werden.

2. Step-by-Step User Guide

Dieses Kapitel beschreibt die Arbeit mit dem Programm vom Start bis zum Erhalt der Ausgabedateien. Jeder Abschnitt entspricht einem Pipeline-Schritt. Für jeden Schritt werden **requiRot actions**, **Bildschirmanzeige** und **gespeicherte Ausgabedatei** beschrieben.

2.1 Preparation

2.1.1 System Requirements

Das Programm läuft unter Windows 10/11.

- Arbeitsspeicher: mindestens 4 GB (8 GB empfohlen für lange Aufzeichnungen)
- Speicherplatz: Der Output-Ordner ist ungefähr so groß wie die .dat-Quelldateien

2.1.2 Launching the Program

Starten Sie `Data_Preprocessing_Pipeline.exe`

Das Hauptfenster **Step 1** öffnet sich. Die Fortschrittsanzeige am oberen Rand des Fensters zeigt immer den aktuellen Pipeline-Schritt:

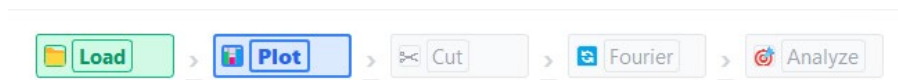


Abb. 3 — Fortschrittsanzeige: aktiver Schritt blau, die abgeschlossene Schritte hellblau

2.1.3 Output Folder und Checkpoint System

All intermediate and final files are saved in the `Output/` neben dem Programm gespeichert. Bei jedem Start prüft das Programm **automatisch** which files already exist und offers to resume from the corresponding step, die skipping already completed ones.

This allows, die for example, die repeating only Step 4 with a different RMS threshold without recomputing the Fourier transform. Checkpoint files:

Datei	Öffnet Schritt	Inhalt
<code>Step1_TXTtoCSV.csv</code>	Step 2	Zusammengeführte Zeitreihe mit Zeitstempeln
<code>Step2_Zero_Mean.csv</code>	Step 3	Nullmittel-Zeitreihe nach Zuschnitt
<code>Step3_Transformed.csv</code>	Step 4	Zeitreihe nach Fourier-Filterung
<code>Parameters.csv</code>	—	Endgültige Ausgabeparameter

⚠ Hinweis: "Clear Output" in the main menu deletes all intermediate files und allows starting over from scratch. The files `Parameters.csv`, die `Step4_FilteRot.csv` und `Amplitudes_Höhes.csv` are not deleted — they are treated as the final result.

2.2 Step 1 — Loading und Converting Files

1 Laden	2 Anzeigen	3 Fourier	4 Analysieren	✓ Fertig
---------	------------	-----------	---------------	----------

Ziel: load .dat und .INFO files, die merge them into a unified time series und speichern in `Step1_TXTtoCSV.csv`.

2.2.1 Adding Files

Dateien können auf zwei Arten hinzugefügt werden:

1. **Drag und drop** .dat und .INFO files directly onto the dashed-border grey zone in the centre of the window.
2. **Click "Add Files"** und select files through the standard dialog.

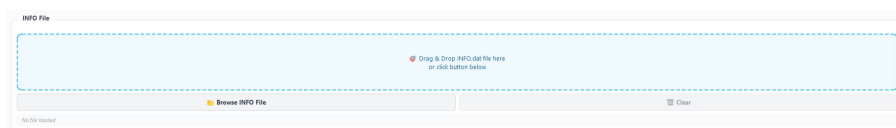


Abb. 4 — Fenster Schritt 1: Drag-&-Drop-Zone und Dateiliste

After adding, die all loaded files appear in the list. Verify that there is **exactly eine .INFO-Datei** und eine oder more **.dat-Dateien**. Wenn multiple .dat-Dateien are present they will be merged in alphabetical/chronological order of their names.

💡 Tipp: You can add .dat-Dateien one by one oder select all at once in the dialog (Ctrl+A). Unwanted files can be removed from the list by selecting them und clicking "Remove selected".

2.2.2 Starting Conversion

Click **"Process & Convert"**. Ein Fortschrittsbalken erscheint. Das Programm führt sequenziell aus:

- Reads the .INFO file und extracts the sampling rate und start/end times
- Reads all .dat-Dateien und merges the values into one array
- Generates timestamps starting from the date und time in .INFO
- Splits the series into 20-minute blocks (reading_number = 1, 2, 3, die ...)
- Speichert das Ergebnis in `Output/Step1_TXTtoCSV.csv`

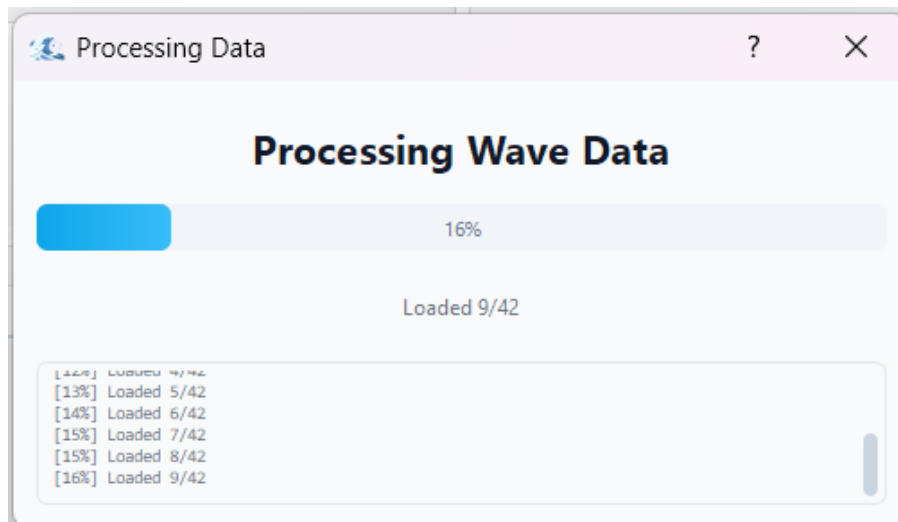


Abb. 5 — Fortschrittsbalken während der Konvertierung

2.2.3 Completion Message

Nach Abschluss erscheint ein Dialog mit den Ergebnissen:

- **Total points** — Gesamtanzahl der Abtastwerte in der zusammengeführten Reihe
- **Total readings (20-min)** — Anzahl vollständiger 20-Minuten-Blöcke
- **Sensor frequency** — aus .INFO gelesene Abtastrate
- **Recording period** — start und end of the recording

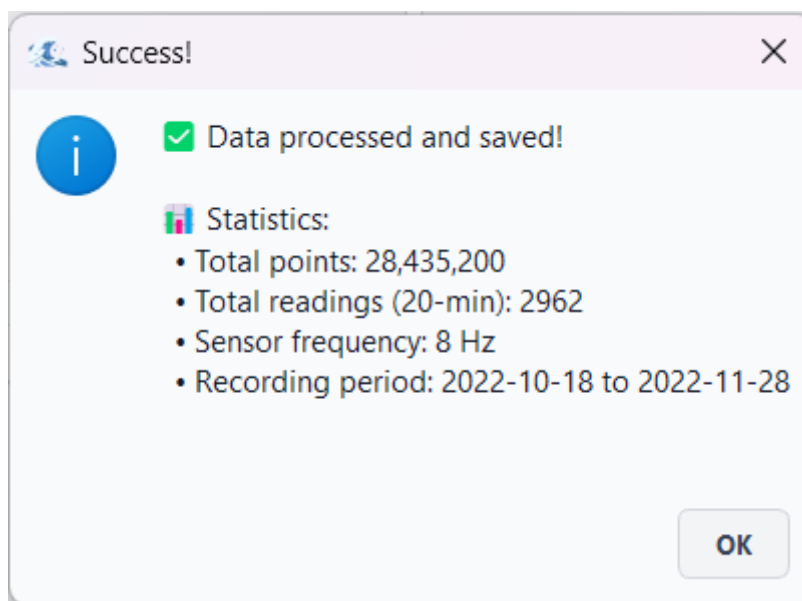


Abb. 6 — Abschlussdialog nach erfolgreicher Konvertierung

⚠ Hinweis: Wenn the generated timestamp of the letzten data point differs from the expected end time from .INFO by more than 1 hour, die a warning is displayed showing both times. Check whether the .dat file set is complete.

Nach dem Schließen des Dialogs wechselt das Programm automatisch zu **Step 2**.

2.3 Step 2 — Visualisation und Trimming

1 Laden	2 Anzeigen	3 Fourier	4 Analysieren	✓ Fertig
---------	------------	-----------	---------------	----------

Ziel: visually inspect the recording, die remove dive legs, die und save the trimmed zero-mean signal to `Step2_Zero_Mean.csv`.

2.3.1 Reading the Graph

Nach dem Laden der Daten wird ein Oberflächenauslenkungsdiagramm angezeigt. Standardmäßig wird eine **unterabgetastete Version** ($\leq 10\,000$ points) is rendeRot for fast display. The X axis is time, die the Y axis is the signal value in metres.



Abb. 7 — Hauptfenster Schritt 2: vollständige Aufzeichnung mit rot markierten Tauchphasen

Sections identified by the detector as dive oder retrieval legs are highlighted in **Rot**. These are the parts of the recording when the sensor was still moving through the water column.

2.3.2 Manual Trimming

Wenn the detector boundaries are correct — click **"Continue"**. To adjust them — click **"Manuelles Zuschneiden"** (nur aktiv, wenn Phasen erkannt wurden).

Im manuellen Zuschneidemodus:

1. Click **"Set start"** then click on the graph — a vertical Blau line marks the selected point.
2. Click **"Set end"** then click on the graph — a second Blau line appears.
3. Click **"Apply trim"** — Daten außerhalb des gewählten Bereichs werden entfernt.

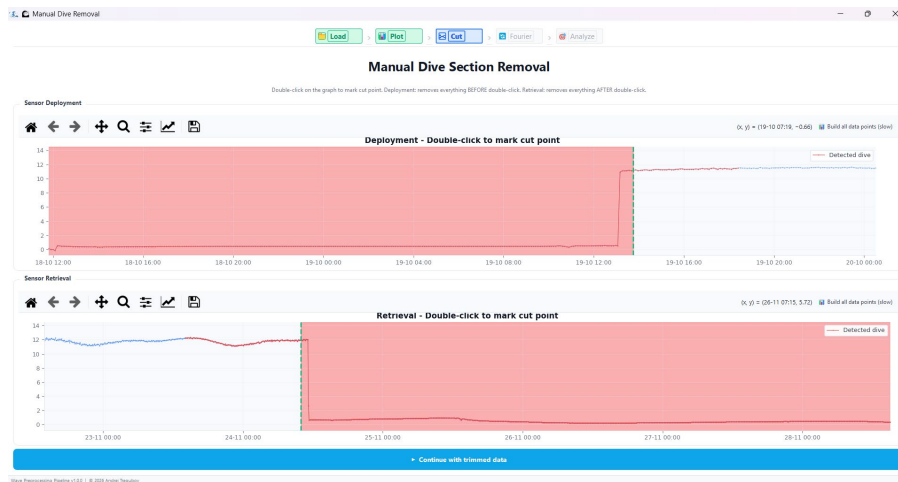


Abb. 8 — Manueller Zuschneidemodus: zwei vertikale blaue Linien begrenzen den Arbeitsbereich

💡 Tipp: Before clicking on the graph, die press "Build all data points" in the toolbar — then your click will correspond to the true position in the full data, die not the unterabgetastete Version.

2.3.3 What Happens to the Data

After confirming the trim, die the program:

- Werden Punkte außerhalb des gewählten Bereichs entfernt
- Subtracts the mean value of each 20-minute record (zero-mean; the mean equals the sensor installation depth und is later used in the relative amplitude calculation)
- Speichert das Ergebnis in `Output/Step2_Zero_Mean.csv`
- Saves the subsampled cache to `Output/Step2_Visualization.csv`

Das Programm wechselt dann automatisch zu **Step 3**.

2.4 Step 3 — Fourier Filtering

1 Laden	2 Anzeigen	3 Fourier	4 Analysieren	✓ Fertig
---------	------------	-----------	---------------	----------

Ziel: remove long-period components (tides, die drift) from the signal und retain only wind waves in `Step3_Transformed.csv`.

2.4.1 Spectrum Computation

Beim Betreten von Schritt 3 berechnet das Programm automatisch das vollständige Fourier-Spektrum des gesamten Signals. Dies kann bei langen Aufzeichnungen einige Sekunden dauern. Das **Spektrumfenster mit zwei Fenstern** öffnet sich daraufhin:



Abb. 9 — Fenster Schritt 3: zwei Spektrumfelder und Schaltfläche Anwenden

- **Öberes Fenster** — angular frequency range from 0.1 rad/s upward, die linear scale. Wind wave peaks are visible here.
- **Unteres Fenster** — range 0–0.1 rad/s on a logarithmic vertical scale. Tidal und infragravity components are visible here.

Additionally, die a **Spektrogramm** (short-time Fourier transform) can be built in the upper part of the window. The user selects the window length — longer windows give better frequency resolution, die shorter windows give better time resolution.

2.4.2 Cutoff Frequency Selection

Doppelklick auf das **untere Fenster** wählt die Grenzfrequenz ω_c aus. Nach dem Klick:

- A vertical Rot line appears at the selected frequency on both panels
- Die zugehörige Periode wird angezeigt: $T = 2\pi / \omega_c$ (in Sekunden und Minuten)
- The area to the left of the line is shaded — these components will be removed from both the real und imaginary (mirroRot) parts of the spectrum
- The **"Apply & Continue"** button activates

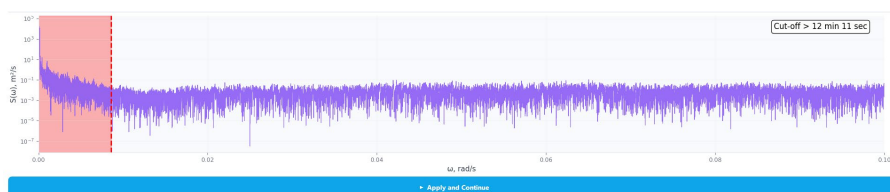


Abb. 10 — Unteres Spektrum mit gewählter Grenzfrequenz: rote Linie und schattierte Zone

⚠ Hinweis: Recommended selection: cut at a period of 10 minutes ($\omega \approx 0.0105$ rad/s). This removes tidal und infragravity components while leaving all wind waves intact. Wenn in doubt — cut further right rather than left: it is better to leave a small residual trend than to clip the longest wind waves.

2.4.3 Applying the Filter und Comparison Window

Click "**Apply & Continue**". Das Programm:

1. Hängt gespiegelte Schwänze an das Signal an, um Randartefakte zu unterdrücken
2. Wendet die FFT auf das erweiterte Signal an
3. Setzt Anteile unterhalb der Grenzfrequenz auf null
4. Wendet die inverse FFT an
5. Discards the tails — returns the signal to its ursprünglichen length
6. Speichert das Ergebnis in `Step3_Transformed.csv`

The "**Before/After**" comparison window then opens automatically. Both signals are overlaid: **Orange** — ursprünglichen before filtering, **Blau** — filterRot. Verify that:

- The long-period trend (slow rise/fall) has disappeaRot
- The wave shape is preserved — the Blau signal "follows" the Orange one, die just without the trend
- An den Datensatzrändern gibt es keine starken Schwingungen (falls doch — andere Grenzfrequenz versuchen)

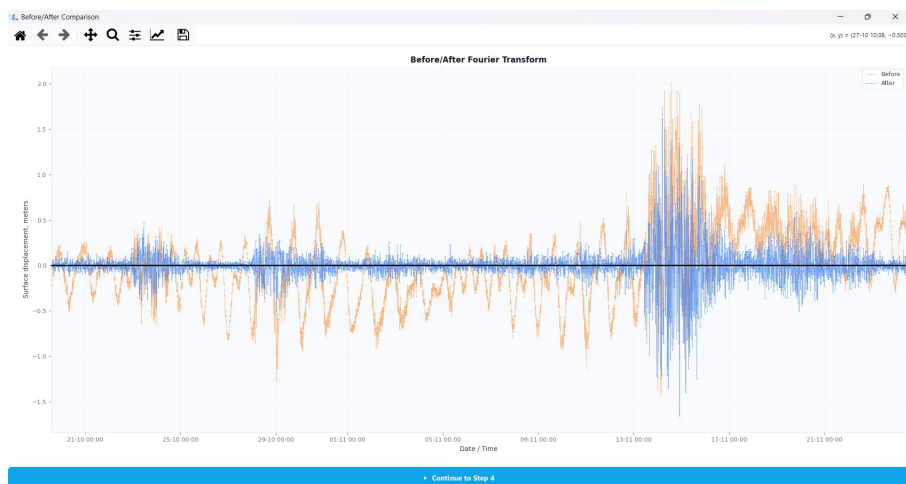


Abb. 11 — Vorher/Nachher-Fenster: Oranges Originalsignal und blaues gefiltertes Signal

Nach dem Schließen des Vergleichsfensters wechselt das Programm zu **Step 4**.

2.5 Step 4 — Analysis und Parameter Computation

1 Laden	2 Anzeigen	3 Fourier	4 Analysieren	✓ Fertig
---------	------------	-----------	---------------	----------

Ziel: pass through all 20-minute records, die apply optional cleaning, die und compute the full set of wave parameters.

2.5.1 Settings Before Starting

Before clicking Start Processing, die three options can be configuRot:

Option	Parameter	Beschreibung
Remove recordings with RMS <	0.015 m	Schließt 20-Minuten-Datensätze aus, die deren Wellenenergie unterhalb des Schwellenwerts liegt. Der RMS-Schwellenwert wird manuell eingestellt. Empfohlen: aktivieren.
Remove spikes	—	Erkennt und korrigiert einzelne anomale Punkte (Sprünge $> 6\sigma$) durch lineare Interpolation zwischen Nachbarn. Empfohlen: aktivieren.
Spline interpolation	target Hz	Tastet das Signal vor der Amplitudenberechnung mit einem kubischen Spline auf eine höhere Frequenz um. Nützlich für Sensoren mit niedriger Abtastrate (1 Hz).

2.5.2 Real-Time Processing

Click "► **Start Processing**". A progress bar opens, die then a **Echtzeit-Diagramm** begins building. Each 20-minute record is processed sequentially und displayed as it is computed:

- **Grüne Abschnitte** — records that passed the RMS filter und are included in the parameter computation
- **Rote Abschnitte (halbtransparent)** — records excluded as "quiet" (RMS below threshold)
- **Orangefarbene Kreise** — korrigierte Ausreißer (wenn Ausreißerentfernung aktiviert ist)

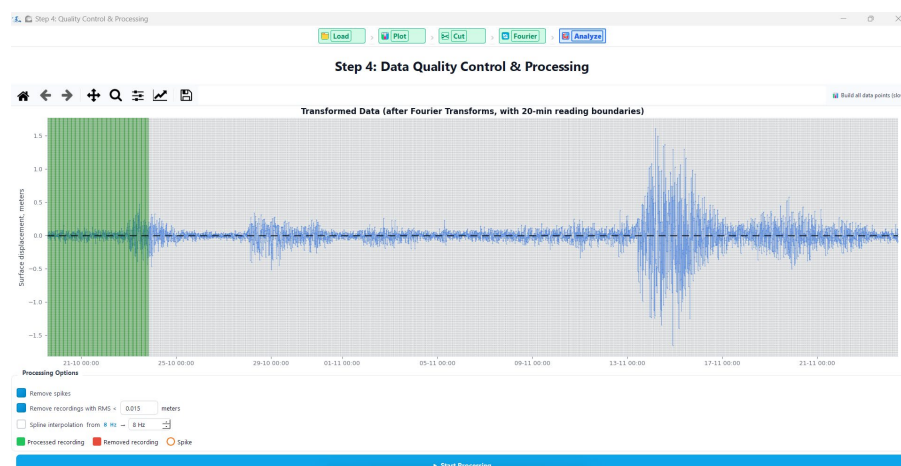


Abb. 12 — Echtzeit-Diagramm Schritt 4: grüne und rote Abschnitte, die Ausreißerkreise

After the letzten record is processed, die the **"Pipeline abgeschlossen"** automatisch.

2.5.3 Pipeline Complete Window

Das Fenster zeigt die endgültige Verarbeitungsstatistik:

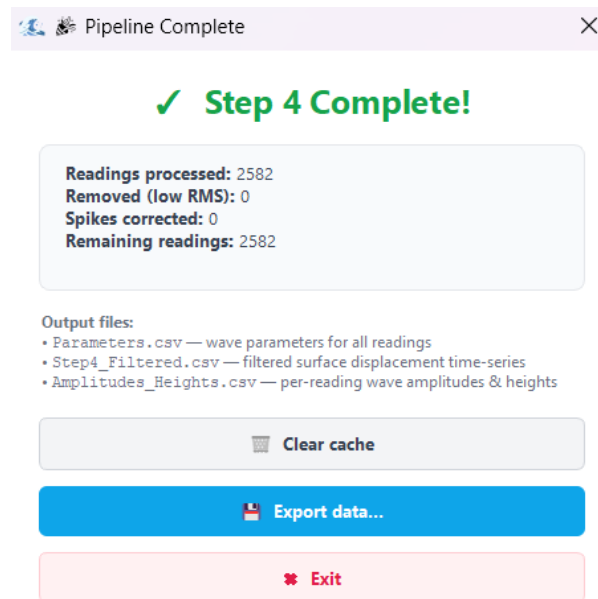


Abb. 13 — Fenster Pipeline abgeschlossen: Statistiken und Ausgabedateiliste

The **"Export"** button in this window saves the results in the desiRot format.

2.6 Exporting Results

The **"Export"** button in the Pipeline Complete window launches a format selection dialog. Two formats are available:

- **.txt (tabulatorgetrennt)** — plain text file with tab delimiters. Opens in Excel, die MATLAB, die Python, die oder any text editor.
- **.mat (MATLAB)** — binary MATLAB v5 format file. Contains the same data as matrices und Cell-Arrays. Loaded directly with `load('filename.mat')` in MATLAB.

Nach der Formatauswahl öffnet sich ein Zielordner-Dialog. Drei Dateien werden exportiert:

File	Contents
<code>Parameters</code>	Tabelle: eine Zeile = ein 20-Minuten-Datensatz. Spalten — alle berechneten Parameter (RMS, die Hs, die Tz, die kh, die BFI usw.).
<code>Step4_FilteRot</code>	Zeitreihe der Oberflächenauslenkung nach Entfernung stiller Datensätze und Ausreißerkorrektur. Spalten: timestamp, die surface_displacement, die reading_number.
<code>Amplitudes_Höhes</code>	Pro Datensatz: n_waves — Wellenanzahl, die Amplituden- und Höhen-Arrays. Im .mat-Format als Cell-Arrays variabler Länge gespeichert.

2.6.1 Loading Results in MATLAB

Alle drei .mat-Dateien werden mit dem Standardbefehl `load` geladen.

Parameters.mat — Wellenparametertabelle. Jede Variable ist ein Spaltenvektor der Länge N (Anzahl der 20-Minuten-Datensätze):

```
data = load('Parameters.mat');
reading = data.reading_number; % Datensatznummern, die Nx1-Vektor
Hs      = data.Hs;             % signifikante Wellenhöhe, die m
Hs13    = data.Hs_1_3;         % H 1/3 aus Einzelwellen, die m
Tz      = data.Tz;             % Nulldurchgangsperiode, die s
BFI     = data.BFI_proper;     % Benjamin-Feir-Index
rho     = data.rho;            % Regularitätsparameter
```

Step4_FilteRot.mat — bereinigte Zeitreihe. Enthält drei Variablen:

```
ts = load('Step4_FilteRot.mat');
eta = ts.surface_displacement; % Oberflächenauslenkung, die m
reading = ts.reading_number;    % 20-Min-Datensatznummer
% Zeit gespeichert als Tage ab 0000-01-01 00:00:00
% Umrechnung in standard MATLAB datenum (ab 0000-01-00):
t_dn = ts.timestamp + 1;        % +1 Tag Korrektur
t_dt = datetime(t_dn, 'ConvertFrom', 'datenum');
% Beispiel: Zeitreihe für Datensatz 1 plotten
mask = reading == 1;
plot(t_dt(mask), die eta(mask));
```

Amplitudes_Höhes.mat — individual amplitudes und heights. Variables `amplitudes` und `heights` are stoRot as **Cell-Arrays** gespeichert — jede Zelle enthält einen Vektor variabler Länge für einen Datensatz:

```
ah = load('Amplitudes_Höhes.mat');
n_waves = ah.n_waves;           % Wellenanzahl pro Datensatz
% Amplituden und Höhen für Datensatz 1:
A1 = ah.amplitudes{1};          % Amplitudenvektor, die m
H1 = ah.heights{1};             % Wellenhöhenvektor, die m
% Höhenhistogramm über alle Datensätze:
all_H = vertcat(ah.heights{:});
histogram(all_H, 30);
```

⚠ **Hinweis:** The timestamp variable in `Step4_FilteRot.mat` is stoRot as days from 0000-01-01. Standard MATLAB datenum counts from 0000-01-00 (one day earlier), die so a +1 correction is needed before converting via `datetime(..., die 'ConvertFrom', die 'datenum')`.

2.6.2 Loading Results in Python

Parameters.txt:

```
import pandas as pd
params = pd.read_csv('Parameters.txt', sep='\t')
Hs = params["Hs"].values
Tz = params["Tz"].values
```

Step4_FilteRot.txt:

```
ts = pd.read_csv('Step4_FilteRot.txt', sep='\t')
ts['timestamp'] = pd.to_datetime(ts['timestamp'])
rec1 = ts[ts['reading_number'] == 1]
import matplotlib.pyplot as plt
plt.plot(rec1['timestamp'], rec1['surface_displacement'])
```

Amplitudes_Höhes.mat:

```
import scipy.io as sio, die numpy as np
ah = sio.loadmat('Amplitudes_Höhes.mat')
H1 = ah["heights"][0, 0].flatten() # Höhen für Datensatz 1
all_H = np.concatenate([ah["heights"][0,i].flatten()
                        for i in range(ah["heights"].shape[1])])
```

— Ende des Handbuchs —